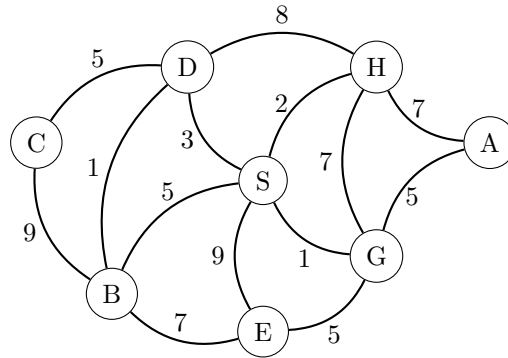


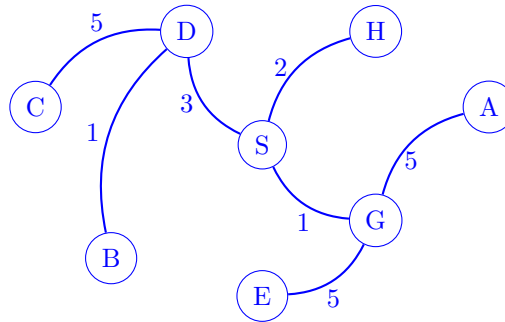
Week 11. Problem set (solutions by Karim Khabibrakhmanov)

- Run Prim-Jarník algorithm [CLRS, §21.2] on the following graph $\mathcal{G}$, starting at vertex S , and answer the questions below.



- Draw a minimum spanning tree of the graph $\mathcal{G}$.

Answer:



- Is the minimum spanning tree for graph $\mathcal{G}$ unique?

Answer:

Yes, because when executing the **Prim-Jarník algorithm**, there was no ambiguity in the choice of edges. All edges with the same weights were either uniquely added to the **MST**, or uniquely excluded:

- The vertex **G** has two edges with a weight of 5, and they are both included in the spanning tree.
- The vertex **H** has two edges with weight 7, and they are both not included in the spanning tree.

- Assuming that the algorithm is using Binary Heap implementation [CLRS, §6] of a priority queue, show the state of the priority queue after each iteration of the algorithm (i.e. after adding each new vertex to the MST). The graph contains 8 vertices, which means that your solution must provide 8 states of the Binary heap. Each heap state must be represented as an array.

For example, a binary min-heap containing key-value pairs $\langle 3, a \rangle, \langle 2, b \rangle, \langle 1, c \rangle, \langle 4, d \rangle$ may be represented as follows:

1, c	3, a	2, b	4, d
------	------	------	------

Answer:

- | | | | | |
|------|------|------|------|------|
| 1, G | 2, H | 3, D | 5, B | 9, E |
|------|------|------|------|------|
- | | | | | |
|------|------|------|------|------|
| 2, H | 5, B | 3, D | 5, E | 5, A |
|------|------|------|------|------|
- | | | | |
|------|------|------|------|
| 3, D | 5, B | 5, A | 5, E |
|------|------|------|------|
- | | | | |
|------|------|------|------|
| 1, B | 5, E | 5, A | 5, C |
|------|------|------|------|
- | | | |
|------|------|------|
| 5, C | 5, E | 5, A |
|------|------|------|

6.	<table><tr><td>5, A</td><td>5, E</td></tr></table>	5, A	5, E
5, A	5, E		
7.	<table><tr><td>5, E</td></tr></table>	5, E	
5, E			
8.	<table><tr><td></td></tr></table>		

2. Suppose that T is the minimum spanning tree for graph $\mathcal{G}$. How difficult is it to update the minimum spanning tree after adding a new vertex and its incident edges to $\mathcal{G}$.

- (a) Describe your algorithm (write pseudocode).

Answer:

```

1  UpdateSpanningTree(old T, new vertex, vector<Edge*> allAdjToVertex):
2      T->addVertex(vertex)
3
4      for (Edge* edge : allAdjToVertex):
5          T->addEdge(edge)
6
7      T = Kruskal-Algorithm(T)
8      return T

```

- (b) Provide the asymptotic time complexity for your algorithm.

Answer: $\Theta(|V| \cdot \log(|V|))$

- (c) Justify the asymptotic time complexity (1–3 sentences).

Justification:

Adding a vertex takes $\Theta(1)$. Adding all sides will take $\Theta(|V|-1)$ in the worst case, if the new vertex is connected to each old one. The most expensive is the Kruskal's algorithm, which takes $\Theta(|V| \cdot \log(|V|))$, but since the maximum number of edges in the algorithm is the old $|V|-1$ and the new ones added in the worst case $|V|-1$ give us $\Theta((2 \cdot |V|-2) \cdot \log(|V|)) \approx \Theta(|V| \cdot \log(|V|))$. Therefore, $\Theta(1 + |V| - 1 + |V| \cdot \log(|V|)) \approx \Theta(|V| \cdot \log(|V|))$

3. Suppose that all edge weights in a graph are *real numbers* uniformly distributed in the range from 0 to k .

- (a) Which of the MST algorithms, Kruskal's or Prim's (possibly with minor modifications) can run faster on such a graph?

Answer: Kruskal's algorithm, because we can implement sorting of the sides' weights in linear time. This gives us an advantage over Prim's algorithm.

- (b) Which modification is required to make the algorithm run faster? Justify your answer (2–3 sentences).

Answer: Bucket sort

Justification:

Since the edge weights are evenly distributed in the range $[0, k]$, we can safely use Bucket sorting without worrying about all the elements falling into one segment. This allows the edges to be sorted in $\Theta(|E|)$ time rather than $\Theta(|E| \cdot \log|E|)$, which makes Kruskal's algorithm generally faster.

- (c) What is the *average* case time complexity of the modified algorithm? Justify your answer (1—2 sentences).

Answer: $\Theta(|E| + |V| + |E| \cdot A(|V|)) \approx \Theta(|E| \cdot A(|V|))$, where $A()$ is Ackermann function.

Justification:

To begin with, we use Bucket sort to sort the list of our edges in $\Theta(|E|)$. Next, the Kruskal algorithm is applied, which takes $\Theta(|V| + |E| \cdot A(|V|))$, where $A()$ is Ackermann function. As a result, we add them up and get $\Theta(|E|) + \Theta(|V| + |E| \cdot A(|V|)) = \Theta(|E| + |V| + |E| \cdot A(|V|)) \approx \Theta(|E| \cdot A(|V|))$.

References

- [CLRS] Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C., 2022. *Introduction to algorithms, Fourth Edition*. MIT press.
- [GTG] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser. *Data Structures and Algorithms in Java*. WILEY 2014.